

A COOKBOOK FOR BACKEND DEVELOPERS

# Mailpit Cookbook

Capture, view, and test the email your app sends — locally and in CI — with Mailpit, the modern MailHog successor.

Mailpit

SMTP :1025

Web UI :8025

REST API

Docker

Nodemailer

Django / smtplib

CI testing

---

Written for engineers who send email from Node.js, Python, or Laravel apps and need a safe way to test it.

From zero to wiring, viewing, and asserting emails in automated tests — without sending a single real message.

Edition 2026 · run it · wire it · test it

# Table of Contents

---

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>How to Use This Book</b>                           | <b>3</b>  |
| What Mailpit is                                       | 3         |
| The mental model: a trap, not a relay                 | 3         |
| <b>Part 1 · Why a Local Mail Server</b>               | <b>4</b>  |
| 1.1 The problem with testing email                    | 4         |
| 1.2 The options, compared                             | 4         |
| 1.3 Why not MailHog?                                  | 4         |
| <b>Part 2 · Running Mailpit</b>                       | <b>6</b>  |
| 2.1 The two ports                                     | 6         |
| 2.2 Docker (recommended)                              | 6         |
| 2.3 Binary / Homebrew                                 | 7         |
| 2.4 Configuration you'll actually touch               | 7         |
| <b>Part 3 · Wiring Your App to Mailpit</b>            | <b>8</b>  |
| 3.1 Node.js (Nodemailer)                              | 8         |
| 3.2 Python (smtplib and Django)                       | 8         |
| 3.3 Laravel (for the PHP side of your stack)          | 9         |
| 3.4 The universal rule                                | 9         |
| <b>Part 4 · The Web UI &amp; Features</b>             | <b>11</b> |
| 4.1 Reading a message                                 | 11        |
| 4.2 Search                                            | 11        |
| 4.3 The checks that catch real bugs                   | 11        |
| 4.4 Tagging and chaos                                 | 12        |
| <b>Part 5 · The REST API &amp; Automation</b>         | <b>13</b> |
| 5.1 The endpoints you'll use                          | 13        |
| 5.2 Reading messages from code                        | 13        |
| 5.3 The reset-between-tests trick                     | 14        |
| 5.4 Search-driven assertions                          | 14        |
| <b>Part 6 · Testing Email End-to-End</b>              | <b>16</b> |
| 6.1 The pattern                                       | 16        |
| 6.2 Node (Jest / Vitest + Supertest)                  | 16        |
| 6.3 Python (pytest)                                   | 16        |
| 6.4 Mailpit in CI                                     | 17        |
| <b>Part 7 · Production, Staging &amp; Cheat Sheet</b> | <b>19</b> |
| 7.1 The environments, and what mail goes where        | 19        |
| 7.2 Staging without spamming real people              | 19        |
| 7.3 Security notes for shared instances               | 19        |
| 7.4 Cheat sheet                                       | 20        |

# How to Use This Book

Your app sends email — welcome messages, password resets, receipts, the webhook digests from the Queues cookbook. During development and testing you need to *see* those emails and *assert* on them, but you absolutely must not send real mail to real people from your laptop or your CI pipeline. **Mailpit** solves exactly this: it's a tiny local mail server that captures everything your app sends, shows it in a clean web UI, and exposes an API so your tests can check what was sent.

This is a short, practical cookbook. By the end you can run Mailpit, point any app at it (Node, Python, Laravel), explore captured mail in the UI, and write automated tests that assert "a password-reset email was sent to this address with this link" — all without a single real message leaving your machine.

## What Mailpit is

Mailpit is a single Go binary (or Docker image) that runs two things at once:

- an **SMTP server** on port **1025** that accepts mail from your app but never delivers it onward — it just captures it;
- a **web UI + REST API** on port **8025** where you (and your tests) read the captured messages.

It is the modern successor to **MailHog** (which is now unmaintained) — same idea, but faster, actively developed, and with more features: full-text search, HTML/link/spam checking, attachments, real-time updates, message tagging, and an API built for automated testing.

### NODE.JS MENTAL MODEL

Think of Mailpit as a fake SMTP endpoint you drop in place of SendGrid/SES in dev. Instead of pointing Nodemailer (or Django's mail backend) at a real mail provider, you point it at `localhost:1025`. Your app's mail code runs unchanged and thinks it sent real email; Mailpit quietly catches it and lets you inspect it at `http://localhost:8025`. It's the mail equivalent of hitting a local mock server instead of a live API.

## The mental model: a trap, not a relay

The one thing to internalize: Mailpit is a **trap**. Mail goes in and stops there for you to look at — it does not reach real inboxes. That's the whole point in development (safe) and the reason it must **never** run in production as your real mail path (nothing would ever get delivered). Keep it firmly in the dev/test lane; Part 7 covers what to use in production instead.

# Part 1 · Why a Local Mail Server

Before the setup, be clear on the problem — it explains every design choice Mailpit makes.

## 1.1 The problem with testing email

Email is a side effect that's genuinely awkward to develop against:

- **You can't send real mail in dev/CI.** Spamming real addresses (or your own) on every test run is unacceptable, and using a real provider burns quota, needs credentials, and can get you rate-limited or blocklisted.
- **You need to *see* the rendered result.** Does the HTML look right? Is the reset link correct? Are the headers and attachments there? A log line saying "email sent" tells you nothing.
- **Your tests need to assert on it.** "After signup, a welcome email is sent to the user's address containing a verification link" is real behavior worth a test — but only if the test can inspect the message.

## 1.2 The options, compared

| Approach                     | See it?     | Assert in tests? | Sends real mail?        | Notes                         |
|------------------------------|-------------|------------------|-------------------------|-------------------------------|
| Log to console               | text only   | clumsy           | no                      | no HTML, no assertions        |
| Write to a file/folder       | sort of     | clumsy           | no                      | framework-specific, no UI     |
| Real provider (SES/SendGrid) | in an inbox | hard             | <b>yes</b> <sup>△</sup> | quota, creds, risk            |
| <b>Mailpit</b>               | ✓ web UI    | ✓ REST API       | <b>no</b>               | purpose-built for this        |
| Mailtrap (hosted)            | ✓           | ✓                | no                      | SaaS; good for shared/staging |

Mailpit wins for local + CI because it's free, self-contained, gives you a real UI *and* a testing API, and never sends anything.

## 1.3 Why not MailHog?

MailHog pioneered this pattern and you'll still see it in older tutorials, but it's **effectively unmaintained**. Mailpit is a drop-in modern replacement by the same style of tooling: faster, actively developed, with a nicer UI and a richer API (search, HTML/link/spam checks, tagging, real-time updates). If you're choosing today, choose Mailpit; if you're on MailHog, migrating is mostly changing the image and the ports.

**PRODUCTION TIP**

Standardize Mailpit across your whole team and CI by putting it in your project's `docker-compose.yml` (Part 2). Then every developer's app and every CI run captures mail the same way, with zero per-machine setup and zero risk of a stray real email. "It's in compose" is the difference between a tool one person uses and a team practice.

**WHEN NOT TO USE MAILPIT**

Mailpit is a *development and testing* tool. Don't use it as your production mail path (it delivers nothing), and don't use it to test actual deliverability, spam-folder placement, or how a specific provider renders your mail — for those you need a real provider or a service like Mailtrap's hosted checks. Mailpit answers "did my app send the right message?", not "will Gmail deliver it to the inbox?".

# Part 2 · Running Mailpit

Mailpit is a single binary with no dependencies, so getting it running is quick. The recommended way for a team is Docker Compose, but all the options are here.

## 2.1 The two ports

Whatever way you run it, Mailpit exposes:

| Port | Protocol | For                                        |
|------|----------|--------------------------------------------|
| 1025 | SMTP     | your app sends mail here                   |
| 8025 | HTTP     | web UI <b>and</b> REST API ( /api/v1/... ) |

Remember them as "1025 in, 8025 look."

## 2.2 Docker (recommended)

### SHELL

```
docker run -d --name mailpit -p 1025:1025 -p 8025:8025 axllent/mailpit
# app sends to localhost:1025 ; open http://localhost:8025
```

In a project's `docker-compose.yml`, so the whole team and CI get it for free:

### YAML

```
services:
  mailpit:
    image: axllent/mailpit
    ports:
      - "1025:1025"      # SMTP
      - "8025:8025"      # web UI + API
    environment:
      MP_MAX_MESSAGES: 500           # ring-buffer cap (0 = unlimited)
      MP_DATABASE: /data/mailpit.db  # persist messages across restarts (optional)
      # MP_SMTP_AUTH_ACCEPT_ANY: 1   # accept any SMTP login (handy in dev)
      # MP_SMTP_AUTH_ALLOW_INSECURE: 1 # allow auth without TLS (dev only)
    volumes:
      - mailpit-data:/data
volumes:
  mailpit-data:
```

By default Mailpit keeps messages **in memory** (they vanish on restart, which is usually fine for dev). Set `MP_DATABASE` to a file on a volume if you want them to persist.

## 2.3 Binary / Homebrew

### SHELL

```
# macOS
brew install mailpit
mailpit                                # runs with defaults (1025 / 8025)

# or the install script / a downloaded release binary
mailpit --help                         # every option has a flag and an MP_ env var
```

## 2.4 Configuration you'll actually touch

Mailpit is configured by flags or `MP_`-prefixed environment variables (same names). The ones that come up:

| Setting             | Env var                                                       | Purpose                                    |
|---------------------|---------------------------------------------------------------|--------------------------------------------|
| Max stored messages | <code>MP_MAX_MESSAGES</code>                                  | cap the ring buffer (default keeps recent) |
| Persist to disk     | <code>MP_DATABASE</code>                                      | keep messages across restarts              |
| Listen addresses    | <code>MP_SMTP_BIND_ADDR</code> / <code>MP_UI_BIND_ADDR</code> | change host/port                           |
| UI auth             | <code>MP_UI_AUTH</code>                                       | protect the UI (user:pass)                 |
| SMTP auth           | <code>MP_SMTP_AUTH</code> / <code>..._ACCEPT_ANY</code>       | require or accept-any SMTP login           |

### PRODUCTION TIP

Pin the image tag ( `axllent/mailpit:v1.x` ) in compose rather than floating on latest, so a teammate's `docker compose pull` can't surprise your test suite with behavior changes. Keep `MP_MAX_MESSAGES` bounded so a long-running dev instance doesn't grow forever, and only add `MP_DATABASE` persistence if you actually want mail to survive restarts — in-memory is simpler and usually what dev wants.

### EXERCISE 2.1

Add Mailpit to a project's `docker-compose.yml` with the two ports and a bounded `MP_MAX_MESSAGES`. Bring it up, open `http://localhost:8025`, and send a test message with a one-liner (e.g. `swaks --to a@b.dev --server localhost:1025`, or the small Node/Python snippet from Part 3). Confirm it appears in the UI, then restart the container and note the messages are gone (in-memory) — then add `MP_DATABASE` on a volume and confirm they persist.

# Part 3 · Wiring Your App to Mailpit

The beauty of Mailpit is that your mail code doesn't change — you only point your SMTP settings at `localhost:1025`. Here's how in each stack. In every case: host `localhost`, port `1025`, no TLS, and auth either off or "any".

## 3.1 Node.js (Nodemailer)

### JAVASCRIPT / NODE.JS

```
import nodemailer from "nodemailer";

const transport = nodemailer.createTransport({
  host: process.env.SMTP_HOST ?? "localhost",
  port: Number(process.env.SMTP_PORT ?? 1025),
  secure: false, // no TLS for local Mailpit
  // auth: undefined // Mailpit accepts unauthenticated by default
});

await transport.sendMail({
  from: "app@example.com",
  to: "ahmed@example.com",
  subject: "Welcome!",
  text: "Thanks for signing up.",
  html: "<h1>Welcome</h1><p>Thanks for signing up.</p>",
});
// now visible at http://localhost:8025
```

## 3.2 Python (smtplib and Django)

### PYTHON

```
# raw smtplib
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["From"] = "app@example.com"; msg["To"] = "ahmed@example.com"
msg["Subject"] = "Welcome!"; msg.set_content("Thanks for signing up.")

with smtplib.SMTP("localhost", 1025) as s: # no TLS, no login
    s.send_message(msg)
```

### PYTHON



```
# Django – settings.py (development)
EMAIL_BACKEND = "django.core.mail.backends.smtp.EmailBackend"
EMAIL_HOST = "localhost"
EMAIL_PORT = 1025
EMAIL_USE_TLS = False
EMAIL_HOST_USER = ""
EMAIL_HOST_PASSWORD = ""
# then: from django.core.mail import send_mail; send_mail(...)
```

### 3.3 Laravel (for the PHP side of your stack)

INI / .ENV

```
# .env
MAIL_MAILER=smtp
MAIL_HOST=localhost
MAIL_PORT=1025
MAIL_USERNAME=null
MAIL_PASSWORD=null
MAIL_ENCRYPTION=null
MAIL_FROM_ADDRESS="app@example.com"
```

### 3.4 The universal rule

| Setting             | Value for Mailpit                             |
|---------------------|-----------------------------------------------|
| Host                | localhost (or mailpit inside Docker networks) |
| Port                | 1025                                          |
| Encryption / TLS    | none                                          |
| Username / password | none (or anything, with accept-any)           |
| From address        | anything you like                             |

#### COMMON MISTAKE

Two wiring gotchas. (1) **TLS on.** Pointing a provider config that expects TLS/SSL at Mailpit's plain port fails to connect — set encryption to none in dev (or enable Mailpit's TLS if you must). (2) **The wrong host inside Docker.** If your app runs in a container too, `localhost` means *that container*, not Mailpit — use the compose **service name** (`mailpit`) as the host, since they share a network. `localhost:1025` only works when the app runs on the host machine.

**PRODUCTION TIP**

Drive the SMTP host/port from **environment variables** (as above), so the exact same code points at Mailpit in dev/CI and at SES/SendGrid in production — you change env, not code. This is the Twelve-Factor mail setup: `SMTP_HOST` / `SMTP_PORT` (or the framework's `MAIL_*`) differ per environment, the app is identical.

**EXERCISE 3.1**

Point your app's mailer at Mailpit via env vars and trigger a real code path that sends mail (e.g. a signup that sends a welcome email). Confirm it lands in the UI with the right subject, HTML, and recipient. Then run your app in a container on the same compose network and switch the host to `mailpit` — proving you understand the localhost-vs-service-name distinction.

# Part 4 · The Web UI & Features

Open `http://localhost:8025` and you get a Gmail-like inbox of everything your app has sent. This part tours what's there — most of it is self-explanatory, but a few features are genuinely useful for catching real bugs.

## 4.1 Reading a message

Each captured message shows:

- **HTML, Text, and Source (raw) views** — flip between the rendered HTML your users see, the plain-text alternative, and the raw MIME source (headers and all). The Source view is where you confirm headers like `From`, `Reply-To`, `Message-ID`, and custom headers.
- **Attachments** — listed and downloadable, so you can verify that generated PDF/CSV actually attached and opens.
- **Headers** — the full set, including any custom `X-` headers your app adds (useful for tags, tracking, or idempotency keys).
- **Recipients** — `To`, `Cc`, `Bcc` (Mailpit captures Bcc, which a real inbox hides — handy for verifying blind copies).

Messages arrive in **real time** (the UI updates live via web socket as your app sends), so you can trigger a flow and watch the mail appear.

## 4.2 Search

Full-text search across subject, body, addresses, and headers, with filters (`from:`, `to:`, `subject:`, `is:unread`, tags). In a busy dev session this is how you find "the reset email I just triggered" among dozens.

## 4.3 The checks that catch real bugs

Mailpit includes automated analyses that surface problems before your users hit them:

| Check             | What it flags                                                        |
|-------------------|----------------------------------------------------------------------|
| <b>HTML check</b> | HTML/CSS features poorly supported by email clients (Outlook, Gmail) |
| <b>Link check</b> | broken or unreachable links in the message                           |
| <b>Spam check</b> | a SpamAssassin-style score (if enabled) so you can lower it          |

The **HTML check** is the standout: email clients render a crippled, inconsistent subset of HTML/CSS, and this flags the constructs likely to break in Outlook or Gmail — the kind of thing you'd otherwise discover from a customer screenshot.

## 4.4 Tagging and chaos

- **Tags:** messages can be auto-tagged (by rules or an `X-Tags` header) so you can group and filter — e.g. tag all `password-reset` mail.
- **Chaos** (fault injection): Mailpit can be told to *randomly* reject or fail a percentage of SMTP deliveries, so you can test that your app's mail-sending code (and its **retry/queue** logic from the Queues cookbook) handles SMTP failures gracefully instead of assuming mail always succeeds.

### PRODUCTION TIP

Make the **HTML check** part of your review habit for any new email template — it's the cheapest way to avoid "the button is invisible in Outlook" bugs. And use **Chaos** when building the mail-sending path: point your queue-backed mailer at a Mailpit configured to fail 20% of deliveries and confirm your jobs retry and eventually succeed, rather than silently dropping mail on the first SMTP hiccup.

### EXERCISE 4.1

Send an HTML email with a deliberately email-unfriendly layout (flexbox, a web font, an external link that 404s) and read the HTML-check and link-check warnings. Add an `X-Tags: welcome` header and confirm the tag appears and is filterable. If you're on the Queues cookbook's mailer, enable Chaos to fail a fraction of sends and watch your retry logic handle it.

# Part 5 · The REST API & Automation

The web UI is for humans; the **REST API** is for your tests and scripts. This is what makes Mailpit more than a viewer — your automated tests can send mail through the app, then ask Mailpit "what got sent?" and assert on it. The API lives on the same `:8025` port under `/api/v1/`.

## 5.1 The endpoints you'll use

| Method & path                                       | Does                                           |
|-----------------------------------------------------|------------------------------------------------|
| GET <code>/api/v1/messages</code>                   | list messages (paginated, newest first)        |
| GET <code>/api/v1/message/{ID}</code>               | full message: HTML, text, headers, attachments |
| GET <code>/api/v1/search?query=...</code>           | search (same syntax as the UI)                 |
| DELETE <code>/api/v1/messages</code>                | delete all (reset between tests)               |
| DELETE <code>/api/v1/messages</code> + ids          | delete specific messages                       |
| GET <code>/api/v1/message/{ID}/part/{PartID}</code> | fetch an attachment                            |
| GET <code>/api/v1/info</code>                       | server info / counts                           |

A listed message includes its `ID`, `From`, `To`, `Subject`, `Created`, `Read`, `Snippet`, and `Tags`; fetch the single message for the full body and headers.

## 5.2 Reading messages from code

### JAVASCRIPT / NODE.JS

```
// Node – list and inspect what the app sent
const res = await fetch("http://localhost:8025/api/v1/messages");
const { messages, total } = await res.json();
const latest = messages[0]; // newest first
const full = await (await fetch(`http://localhost:8025/api/v1/message/${latest.ID}`)).json();
// full.HTML, full.Text, full.Subject, full.To, full.Attachments, full.Headers
```

### PYTHON

```
import requests
BASE = "http://localhost:8025/api/v1"
msgs = requests.get(f"{BASE}/messages").json()["messages"]
full = requests.get(f"{BASE}/message/{msgs[0]['ID']}").json()
# full["HTML"], full["Text"], full["Subject"], full["To"], full["Headers"]
```

## 5.3 The reset-between-tests trick

The key to reliable email tests is a clean slate: before (or after) each test, delete all messages so assertions aren't polluted by earlier ones.

JAVASCRIPT / NODE.JS

```
async function clearMailpit() {
  await fetch("http://localhost:8025/api/v1/messages", { method: "DELETE" });
}
// call in beforeEach()
```

PYTHON

```
def clear_mailpit():
    requests.delete("http://localhost:8025/api/v1/messages")
# call in a pytest fixture
```

## 5.4 Search-driven assertions

Rather than listing everything and filtering in code, let Mailpit's search do the work — assert that a message matching a query exists.

JAVASCRIPT / NODE.JS

```
const q = encodeURIComponent(`to:ahmed@example.com subject:"Reset your password"`);
const { messages } = await (await fetch(`http://localhost:8025/api/v1/search?query=${q}`)).json();
expect(messages.length).toBe(1);
```

### PRODUCTION TIP

Wrap the API in a tiny **test helper** (`mailpit.clear()`, `mailpit.latest()`, `mailpit.find({to, subject})`) once, and reuse it across your whole suite — it keeps tests readable and means the raw HTTP details live in one place. Point the helper's base URL at an env var so the same tests run against a local Mailpit and a CI-service Mailpit without change.

### COMMON MISTAKE

Flaky email tests from two causes: (1) **not clearing** messages between tests, so a stale message from an earlier test satisfies (or breaks) a later assertion — always reset in `beforeEach` / a fixture; (2) **race conditions** — asserting immediately after triggering the send before Mailpit has received it, especially when the mail is sent from a **background job** (Queues cookbook). Poll the API briefly (retry for up to ~1-2s) rather than checking once, so an async send has time to land.

**EXERCISE 5.1**

Write a `mailpit` test helper with `clear()`, `messages()`, `latest()`, and `find({to, subject})` using the REST API. Use it to: clear the inbox, trigger your app's password-reset flow, and assert that exactly one message was sent to the right address, with the reset link present in `full.HTML`. Add a short poll so it works even when the mail is sent from a queue worker.

# Part 6 · Testing Email End-to-End

This is where Mailpit earns its place in your pipeline: real, end-to-end email tests. Your test drives the app through a genuine flow, the app really sends via SMTP, Mailpit captures it, and your test asserts on the captured message — the whole path exercised, nothing mocked.

## 6.1 The pattern

Every email test is the same four beats:

### DIAGRAM

- |                       |                                                       |
|-----------------------|-------------------------------------------------------|
| 1. clear Mailpit      | (DELETE /api/v1/messages)                             |
| 2. do the thing       | (call the endpoint / run the flow that sends mail)    |
| 3. fetch from Mailpit | (GET /messages or /search – poll briefly if async)    |
| 4. assert             | (recipient, subject, body/link, attachments, headers) |

## 6.2 Node (Jest / Vitest + Supertest)

### JAVASCRIPT / NODE.JS

```
import { clearMailpit, findMail } from "../helpers/mailpit";

beforeEach(() => clearMailpit());

test("signup sends a welcome email with a verify link", async () => {
  await request(app).post("/signup").send({ email:
    "ahmed@example.com" }).expect(201);

  const msg = await findMail({ to: "ahmed@example.com", subject: "Welcome" }); //
  polls
  expect(msg).toBeTruthy();
  expect(msg.HTML).toContain("/verify?token=");
  expect(msg.To[0].Address).toBe("ahmed@example.com");
});
```

## 6.3 Python (pytest)

### PYTHON

```
import pytest, requests, time
BASE = "http://localhost:8025/api/v1"

@pytest.fixture(autouse=True)
def clear_mail():
```



```

requests.delete(f"{BASE}/messages")
yield

def find_mail(to, subject, tries=20):
    for _ in range(tries):
        q = f'to:{to} subject:{subject}'
        msgs = requests.get(f"{BASE}/search", params={"query": q}).json()["messages"]
        if msgs:
            return requests.get(f"{BASE}/message/{msgs[0]['ID']}").json()
        time.sleep(0.1)
    return None

def test_password_reset_email(client):
    client.post("/password/forgot", json={"email": "ahmed@example.com"})
    msg = find_mail("ahmed@example.com", "Reset your password")
    assert msg is not None
    assert "/reset?token=" in msg["HTML"]

```

## 6.4 Mailpit in CI

Run Mailpit as a service container in your pipeline so CI captures mail exactly like local dev.

### YAML

```

# GitHub Actions
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      mailpit:
        image: axllent/mailpit
        ports: ["1025:1025", "8025:8025"]
    steps:
      - uses: actions/checkout@v4
      - run: # your test command, with SMTP_HOST=localhost SMTP_PORT=1025

```

The app under test points at `localhost:1025` (from the service), and your tests hit `localhost:8025` — identical to local.

### PRODUCTION TIP

Test the **behavior**, not the exact wording. Assert on the recipient, the presence of a working link/token, the right template being used (a stable marker or an `X-Template` header), and attachments — not the full body text, which changes with copy edits and makes brittle tests. And always test the *negative*: "no email is sent when signup fails validation" is as important as the positive case.

### COMMON MISTAKE

Asserting once, immediately, when the email is sent from a **background job** (the common case — Queues cookbook). The endpoint returns before the worker has sent the mail, so a single immediate check fails intermittently. **Poll** the Mailpit API for a second or two (as `find_mail` does above), and make sure a worker is actually running in the test environment, or the mail never sends at all.

### EXERCISE 6.1

Write two end-to-end tests against a real send-through-SMTP path: (1) signup → welcome email with a verify link; (2) failed signup → **no** email. Use a `clear` fixture and a polling `find_mail` helper. Then add Mailpit as a CI service and get both tests green in your pipeline, driving SMTP host/port from env vars.

# Part 7 · Production, Staging & Cheat Sheet

Mailpit is a dev/test tool — but where it sits relative to real mail, and what replaces it in production, is worth stating plainly. This part closes the loop and leaves you a cheat sheet.

## 7.1 The environments, and what mail goes where

| Environment       | Mail target                                          | Why                                     |
|-------------------|------------------------------------------------------|-----------------------------------------|
| <b>Local dev</b>  | Mailpit ( <code>localhost:1025</code> )              | see everything, send nothing real       |
| <b>CI / tests</b> | Mailpit (service container)                          | assert on captured mail, hermetic       |
| <b>Staging</b>    | Mailpit <i>or</i> a hosted trap (Mailtrap)           | safe preview without mailing real users |
| <b>Production</b> | a real provider (SES, SendGrid, Postmark, Resend...) | actually deliver mail                   |

The only thing that changes between these is **configuration** (SMTP host/port/ credentials via env vars, Part 3) — never your application's mail code.

### WHEN NOT TO USE MAILPIT

Never point production at Mailpit — it captures and **never delivers**, so your users would silently receive nothing. Mailpit also isn't a deliverability tool: it can't tell you whether Gmail will inbox or spam your mail, warm your IP, or handle bounces and unsubscribes. Those are jobs for a real ESP (Email Service Provider). Use Mailpit to verify *what your app sends*; use a provider (and its analytics) to verify *what recipients get*.

## 7.2 Staging without spamming real people

On staging you often want to preview real flows without emailing customers. Two good options: run Mailpit on the staging box (same as dev), or use a **hosted trap** like Mailtrap that gives a shared inbox the team can view. Either way, the principle is the same — a trap, not a relay — so QA can inspect mail without a single message reaching a real recipient.

## 7.3 Security notes for shared instances

If you run Mailpit somewhere others can reach (a staging server), it's capturing potentially sensitive content (reset tokens, personal data), so:

- Put it behind **auth** ( `MP_UI_AUTH` ) and don't expose port 8025 to the public internet.
- Bound retention ( `MP_MAX_MESSAGES` ) and clear it periodically.

- Treat captured mail as real data — it contains live tokens and PII.

## 7.4 Cheat sheet

**Ports:** 1025 SMTP (in) · 8025 UI + API (look). **Image:** axllent/mailpit.

| Task               | How                                                                                                   |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Run (Docker)       | <code>docker run -p 1025:1025 -p 8025:8025 axllent/mailpit</code>                                     |
| Wire an app        | SMTP host <code>localhost</code> (or <code>mailpit</code> in Docker), port <code>1025</code> , no TLS |
| View mail          | <code>http://localhost:8025</code>                                                                    |
| List via API       | <code>GET /api/v1/messages</code>                                                                     |
| One message        | <code>GET /api/v1/message/{ID}</code>                                                                 |
| Search             | <code>GET /api/v1/search?query=...</code>                                                             |
| Clear all          | <code>DELETE /api/v1/messages</code>                                                                  |
| Persist mail       | <code>MP_DATABASE=/data/mailpit.db</code> + a volume                                                  |
| Cap storage        | <code>MP_MAX_MESSAGES=500</code>                                                                      |
| Protect UI         | <code>MP_UI_AUTH=user:pass</code>                                                                     |
| Test SMTP failures | enable <b>Chaos</b>                                                                                   |

**The test loop:** clear → act → fetch (poll if async) → assert on recipient, subject, link/token, attachments, headers.

**The golden rule:** Mailpit is a *trap* for dev/test — real delivery is a provider's job, selected by environment config, with your mail code unchanged.

### YOU'RE SET

With Mailpit in your compose file and a small API helper in your test suite, email stops being the untested, scary corner of your app. Every developer sees exactly what's sent, the HTML check catches client-rendering bugs, and CI asserts the right mail goes out — all without a real message leaving the building. Pair it with the **Queues** cookbook (most mail is sent from jobs) and the **Backend Infra Sprint** plan ties the three together.